

Introduction to Operating Systems

COP 4600

Name and ID: Diya Jain U81673988

Homework #1

Due date: 09/10 at 11:59 pm

Instructions:

1) To complete assignment one, you need to read Chapters 1, 2, and 3 of the textbook.

2) HW must be submitted on typed pdf or word document.

You must do your work on your own. Late work will be not accepted.

Q1. What are the three main purposes of an operating system? **(1 point)**

Answer: The three main purposes of an operating system are:

1. Execute user programs and make solving user problems easier.
2. Make the computer system convenient to use.
3. Use the computer hardware in an efficient manner.

Q2. What is the main advantage for an operating-system designer of using virtual-machine architecture? What is the main advantage for a user? **(1 point)**

Answer:

Main Advantage for an Operating-System Designer of Using Virtual-Machine Architecture:

Virtual Machine Monitor (VMM) allows for the creation of virtual machines where multiple operating systems can run concurrently. This benefits operating-system designers by providing a platform for experimentation, development, and testing of various operating systems in isolated environments. It facilitates research, debugging, and innovation in the field of operating systems without impacting the host system.

Main Advantage for a User:

Users benefit from virtual-machine architecture because it enables them to install and run multiple operating systems on a single physical machine. This is particularly useful for exploration, running legacy applications, or using software that is designed for different operating systems. It provides users with flexibility and the ability to maintain diverse environments without needing separate physical hardware for each OS.

Q3. Describe the actions taken by a kernel to context-switching between processes. **(1 point)**

Answer: Context-switching in a kernel, which allows it to switch between different processes, involves these simple actions:

1. **Save Current State:** The kernel saves the current state of the running process, like its registers and program counter. It's like bookmarking where the process left off.
2. **Load New State:** The kernel loads the saved state of the next process that needs to run. Think of it as opening a saved game, starting from where that process was paused.
3. **Update Memory Maps:** The kernel updates memory maps to ensure that the new process can access the right parts of memory and resources.
4. **Switch Control:** The kernel hands over control to the new process. It's like passing the controller in a video game from one player to another.
5. **Resume Execution:** The new process continues running from where it left off, as if nothing happened.

This way, the kernel can switch between processes smoothly, allowing multiple programs to run on a computer one after another.

Q4. What are the benefits and the disadvantages of each of the following? **(3 points)**

- a. Synchronous and asynchronous buffering

Answer:

Benefits of Synchronous Buffering:

Deterministic Communication: Synchronous buffering ensures that data is transferred at predictable intervals, making it suitable for real-time applications where timing is critical.

Disadvantages of Synchronous Buffering:

Limited Flexibility: Synchronous buffering can be less flexible, as both the sender and receiver must operate in strict coordination, which may not be suitable for all communication scenarios.

Benefits of Asynchronous Buffering:

Flexibility: Asynchronous buffering allows for more flexibility as data can be sent and received independently, making it suitable for a wider range of communication scenarios.

Disadvantages of Asynchronous Buffering

Complex Error Handling: Error handling can be more complex because data can be sent and received at different times, requiring additional mechanisms for error detection and correction.

b. Automatic and explicit buffering

Answer:

Benefits of Automatic Buffering:

Efficiency: Automatic buffering can optimize data transfer and I/O operations, reducing the number of small, frequent data transfers, and improving overall system efficiency.

Disadvantages of Automatic Buffering:

Resource Consumption: Automatic buffering may consume extra memory or system resources to maintain buffers, which could be inefficient in resource-constrained environments.

Benefits of Explicit Buffering:

Control: Explicit buffering provides developers with full control over buffer management, allowing them to optimize data handling for specific use cases.

Disadvantages of Explicit Buffering:

Complexity: Explicit buffering can add complexity to application code, requiring careful handling of buffer allocation and deallocation, which can increase the potential for errors.

In summary, automatic buffering offers simplicity and standardization but may consume extra resources, while explicit buffering provides control and resource efficiency but can be more complex to implement and maintain. The choice between them depends on the specific requirements of the application and the trade-offs that the developer is willing to make.

c. Fixed-sized and variable-sized messages

Answer:

Benefits of Using Fixed-Sized Messages:

Easy System Setup: Setting up the system with fixed-sized messages is straightforward and less complex.

Disadvantages of Using Fixed-Sized Messages:

Programming Complexity: Programming becomes more challenging when dealing with fixed-sized messages due to the restrictions they impose.

Benefits of Using Variable-Sized Messages:

Simplified Programming: Programming becomes easier when using variable-sized messages as they offer more flexibility.

Disadvantages of Using Variable-Sized Messages:

Complex System Setup: Setting up the system with variable-sized messages can be more complex and less straightforward.

Q5. Assume that the following program contains no syntax errors. As it executes it will create one or more processes. **(4 points)**

Simulate the execution of this program and show how processes are created.

What is total number of processes? Show your work.

What will this program print on the screen when it executes?

```
#include<stdio.h>
main()
{
int m=10, n=5, count=1, mult=1;
while(count <3)
{
    if(m != 0)
    {
        m = fork(); n = n+25;
    }
    else
    {
        m = fork(); n = n+20; mult = mult*n;
    }
    printf("n = %d    mult = %d", n, mult);
    count =count + 1;
}
}
```

Answer:

Iteration 1:

- Original parent process (P1) forks into child1 (C1) and remains parent (P1)

Iteration 2:

- P1 forks into child2 (C2) and remains P1
- C1 forks into child3 (C3) and remains C1

Iteration 3:

- No new forks, so 4 processes remain: P1, C1, C2, C3

Iteration 1: 1 process -> 2 processes

Iteration 2: 2 processes -> 4 processes

Iteration 3: Remains 4 processes

The total number of processes are 4.

The program will print the following in the screen.

n=30	mult=1
n=30	mult=1
n=55	mult=1
n=55	mult=1
n=50	mult=50
n=50	mult=50